

Documentacao Completa do Sistema - Agente de Precos @page { size: A4; margin: 20mm 14mm 18mm 14mm; } body { margin: 0; color: #16202a; font-family: "Segoe UI", Arial, sans-serif; font-size: 11.5pt; line-height: 1.45; } h1, h2, h3, h4 { margin: 0 0 10px; line-height: 1.2; } h1 { font-size: 24pt; } h2 { margin-top: 28px; padding-bottom: 6px; font-size: 16pt; border-bottom: 1px solid #d7dee5; } h3 { margin-top: 18px; font-size: 13pt; color: #1c6dd0; } h4 { margin-top: 14px; font-size: 11.5pt; } p { margin: 0 0 10px; } ul, ol { margin: 8px 0 12px 22px; padding: 0; } li { margin: 4px 0; } code { padding: 1px 4px; font-family: Consolas, "Courier New", monospace; font-size: 10pt; background: #f7f9fc; border-radius: 4px; } pre { margin: 10px 0 14px; padding: 12px; white-space: pre-wrap; word-break: break-word; font-family: Consolas, "Courier New", monospace; font-size: 9.5pt; line-height: 1.45; background: #f7f9fc; border: 1px solid #d7dee5; border-radius: 8px; } table { width: 100%; margin: 10px 0 16px; border-collapse: collapse; font-size: 10.5pt; } th, td { padding: 8px 10px; vertical-align: top; text-align: left; border: 1px solid #d7dee5; } th { background: #eaf2fd; } .cover { min-height: 240mm; display: flex; flex-direction: column; justify-content: center; gap: 16px; } .cover-card { padding: 22px 24px; background: linear-gradient(135deg, #f2f7ff, #ffffff); border: 1px solid #d7dee5; border-radius: 16px; } .meta { color: #52606d; font-size: 10.5pt; } .note { margin: 12px 0 16px; padding: 12px 14px; background: #fff7e8; border-left: 4px solid #d97706; border-radius: 8px; } .ok { margin: 12px 0 16px; padding: 12px 14px; background: #ecfdf3; border-left: 4px solid #16a34a; border-radius: 8px; } .grid-2 { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; } .card { padding: 12px 14px; background: #fff; border: 1px solid #d7dee5; border-radius: 10px; } .page-break { page-break-before: always; } Documentacao Completa do Sistema

Projeto: Agente de Precos / Price Monitor Pro

Objetivo do documento: registrar de forma minuciosa a arquitetura, as funcionalidades implementadas, a operacao, os fluxos de negocio, os arquivos-chave e as orientacoes de manutencao futura do sistema.

Versao consolidada em 18/03/2026, baseada no codigo-fonte presente na raiz do projeto, na documentacao existente e nas configuracoes operacionais observadas no ambiente.

Resumo executivo

O sistema processa planilhas de produtos, identifica o canal de cada item, coleta precos por automacao ou HTTP, grava os resultados em planilha, disponibiliza painel web, shell Streamlit e interface de terminal, alem de permitir agendamento, envio de e-mail, WhatsApp e Google Drive.

Fontes usadas nesta consolidacao

- ? README.md
- ? DOCUMENTACAO_COMPLETA.md
- ? App/main.py
- ? ui/server.py
- ? ui_streamlit/app.py
- ? run.ps1, install.py e install.ps1

1. Visao Geral do Sistema

O Agente de Precos e um sistema de automacao operacional focado em leitura de planilhas, coleta de precos em marketplaces e geracao de saida estruturada. A aplicacao foi expandida para funcionar em mais de uma forma de uso: via linha de comando, via API HTTP, via painel HTML/JavaScript e via shell Streamlit com camada de autenticacao.

Na pratica, o projeto deixou de ser apenas um coletor executado por script e passou a ser uma

plataforma local de operacao de pesquisas de preco, com fila de jobs, persistencia de estado, acompanhamento em tempo real e distribuicao automatica de resultados.

1.1 Objetivos atendidos pelo sistema

- ? Receber uma planilha de entrada padronizada.
- ? Identificar, por link ou canal, qual coletor deve ser utilizado.
- ? Executar scraping/coleta e gerar uma planilha de saida estruturada.
- ? Disponibilizar operacao por interface web e por console local.
- ? Suportar execucao manual e automatica por agenda diaria.
- ? Enviar resultados por e-mail, WhatsApp e Google Drive.
- ? Permitir autenticacao de usuarios para acesso ao painel.

1.2 Componentes principais

ComponenteArquivo principalPapel

Coletor principalApp/main.pyLe leitura da planilha, roteamento por canal e geracao do output.

Backend HTTPui/server.pyAPI FastAPI, fila de jobs, scheduler, envios e autenticacao.

Painel webui/templates/* e ui/static/*UX operacional do sistema.

Shell Streamlitui_streamlit/app.pyCasca autenticada que incorpora o painel principal.

Cliente localtools/local_console.pyOperacao sem navegador.

Instalacaoinstall.py e install.ps1Provisionamento do ambiente.

2. O Que Foi Feito no Sistema

- ? Introducao de uma API FastAPI para gerenciar jobs, status, downloads e configuracoes em runtime.
- ? Criacao de um painel web em HTML/CSS/JS com atualizacao em tempo real por SSE e fallback de polling.
- ? Criacao de um shell Streamlit para operacao simplificada e autenticada.
- ? Criacao de um cliente reutilizavel para operar a API local ou remota a partir do terminal.
- ? Implementacao de agendamento diario multiplo com janelas de disparo configuraveis.
- ? Implementacao de envio automatico e manual de planilhas por e-mail.
- ? Implementacao de envio por WhatsApp via Twilio.
- ? Implementacao de upload automatico ou manual para Google Drive.
- ? Implementacao de autenticacao de usuarios com CRUD administrativo.
- ? Separacao de modos de execucao: web, local e hibrido.
- ? Preparacao de deploy em Linux com systemd, Nginx e variaveis de ambiente dedicadas.
- ? Melhorias recentes no instalador para detectar e recriar venv quebrado.

Observacao importante: os instaladores foram corrigidos para nao reportarem sucesso falso quando o ambiente virtual estivesse apontando para um Python removido. Hoje eles detectam ambiente quebrado, movem o venv invalido para backup e recriam o ambiente.

3. Estrutura de Pastas e Responsabilidades

Pasta / arquivoFuncao no sistema

App/Nucleo do scraping, configuracao e coletores.

App/collectors/Implementacoes por canal e modulo especializado.

App/utils/Helpers de navegador, VTEX, formatacao e APIs auxiliares.

ui/Backend FastAPI e frontend HTML embutido.

ui_streamlit/Shell Streamlit que encapsula a aplicacao principal.

tools/Cliente Python reutilizavel e console local.

deploy/server/Artefatos para deploy Linux com systemd e Nginx.
runs/Estado persistente de runtime: token, jobs, usuarios e configuracoes salvas.

4. Nucleo de Coleta: App/main.py

O arquivo App/main.py continua sendo o centro do processamento bruto de planilhas. Ele e responsavel por abrir o arquivo de entrada, encontrar as colunas relevantes, decidir qual coletor sera usado e gravar o arquivo final.

4.1 Responsabilidades principais

- ? Ler input.xlsx ou outro caminho recebido por argumento/variavel.
- ? Normalizar nomes de colunas para tolerar variacoes de acentuacao e rotulos.
- ? Extrair URL visivel ou hyperlink real da celula.
- ? Resolver qual coletor sera usado para cada linha.
- ? Padronizar os campos A prazo, A vista e Parcelamento.
- ? Gerar a aba Output e a aba Consolidado.
- ? Aplicar filtros por quantidade de linhas ou IDs especificos.

4.2 Coletores reconhecidos

CanalEstrategia principal

MagaluPlaywright e suporte a API oficial.

WebcontinentalFluxo HTTP/VTEX.

ZemaSelenium.

MadeiraMadeiraSelenium.

ProbelSelenium.

Mercado LivreCodigo ainda presente, mas documentado como desativado para o projeto principal atual.

5. Configuracao Central via App/config.py

As classes ScraperSettings e ServerSettings organizam a configuracao do sistema e leem automaticamente o arquivo .env. Isso centraliza defaults, validacoes e limites operacionais.

- ? enable_parallel_scraping, max_workers, max_selenium_sessions
- ? selenium_driver_max_tasks, selenium_page_load_timeout e retries de abertura
- ? daily_auto_enabled, daily_run_times, api_token e max_active_jobs
- ? blocos completos para SMTP/Gmail, Twilio/WhatsApp, Google Drive e intervalo de eventos da UI

Beneficio pratico: toda a configuracao sensivel fica fora do codigo e pode ser adaptada entre ambiente local, servidor Linux e futuros modos de publicacao.

6. Backend HTTP: ui/server.py

Este arquivo concentrou a maior parte da evolucao do sistema. Ele deixou de ser apenas um adaptador e se tornou o backend orquestrador do produto.

6.1 Responsabilidades principais

- ? Expor o status de saude da aplicacao.
- ? Manter fila de jobs e execucao serializada/limitada.
- ? Gerenciar scheduler diario.
- ? Persistir configuracoes de email, WhatsApp, Google Drive e agendamento.
- ? Persistir usuarios administrativos e sessoes autenticadas.
- ? Disponibilizar snapshots do estado via /api/events.

? Entregar HTML do painel principal e arquivos de output.

6.2 Estruturas centrais

? Job: dataclass que guarda trigger, status, tempos, caminhos de input/output e status de entregas.

? jobs: dicionario em memoria com o historico recente de execucoes.

? job_queue: fila de jobs aguardando processamento.

? latest_manual_job_id e latest_daily_job_id: referencias para a ultima execucao de cada tipo.

? arquivos persistentes em runs/ para token, usuarios e configuracoes.

6.3 Endpoints implementados

GrupoEndpoints

Saude e overview/api/health, /api/overview, /api/functions/check

Execucao/run, /api/run, /api/jobs/stop, /api/status/{job_id}

Ultimos resultados/api/manual/latest, /api/daily/latest, /api/output/latest

Email/api/email/settings, /api/email/{job_id}/send, /api/email/send/latest

WhatsApp/api/whatsapp/settings, /api/whatsapp/{job_id}/send, /api/whatsapp/send/latest

Google Drive/api/drive/settings, /api/drive/{job_id}/upload, /api/drive/upload/latest

Agenda e eventos/api/daily/settings, /api/events

Credenciais de marketplace/api/marketplace/credentials, /api/marketplace/credentials/magalu

Usuarios administrativos/api/admin/users/authenticate, /api/admin/users/logout,

/api/admin/users, /api/admin/users/{username}

Frontend e download/, /status/{job_id}, /results/{job_id}, /download/latest, /download/{job_id}

7. Interface Web e Streamlit

O frontend principal do sistema nao e Streamlit puro. O painel operacional e um frontend HTML/CSS/JavaScript servido pela propria API. O Streamlit atua como shell, camada de autenticao e painel administrativo adicional.

7.1 Arquivos da interface

? ui/templates/index.html, status.html e results.html

? ui/static/style.css e ui/static/app.js

? ui_streamlit/app.py

7.2 Atualizacao em tempo real

- A UI abre um canal EventSource para /api/events.
- O backend envia snapshots periodicos com contadores e informacoes do job atual.
- Se o canal cair, a UI reconecta automaticamente.
- Se SSE falhar, permanece o fallback de polling para resiliencia.

7.3 Fluxo de autenticao no Streamlit

- Se PASSWORD_AUTH_ENABLED=1, o usuario precisa autenticar.
- O login principal ocorre via endpoint /api/admin/users/authenticate.
- O Streamlit valida /api/health e /api/overview antes de incorporar a UI principal.
- Usuarios admin podem cadastrar, editar e excluir usuarios via a propria interface.

8. Interface Local por Console

A pasta tools/ abriga a vertente operacional sem navegador.

? tools/pm_client.py: cliente HTTP reutilizavel para a API.

? tools/local_console.py: menu interativo para operacao humana.

? run_local_console.ps1: atalho PowerShell.

Esse modo suporta health, overview, inicio de jobs, consulta de status, espera por conclusao, download do output, envio por e-mail e consulta da agenda diaria.

9. Agendamento Diario e Fila de Jobs

? DAILY_RUN_TIMES recebe lista CSV no formato HH:MM.

? O padrao operacional e 06:00,12:00,18:00.

? Cada slot diario gera um job automatico identificavel por trigger.

? O backend evita reenfileirar o mesmo slot mais de uma vez no mesmo dia.

? Os jobs percorrem estados como QUEUED, RUNNING, DONE, FAILED, STOPPED e CANCELED.

10. Canais de Entrega Implementados

10.1 E-mail

O sistema suporta envio automatico e manual do output.xlsx. Variaveis relevantes incluem SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_SENDER, GMAIL_USER, GMAIL_APP_EMAIL_RECIPIENTS e EMAIL_SEND_ON.

10.2 WhatsApp via Twilio

Variaveis relevantes incluem TWILIO_ACCOUNT_SID, TWILIO_AUTH_TOKEN, TWILIO_WHATSAPP_FROM, WHATSAPP_RECIPIENTS, WHATSAPP_SEND_ON e APP_PUBLIC_BASE_URL.

10.3 Google Drive

O upload para Google Drive usa service account com GOOGLE_DRIVE_CREDENTIALS_FILE, GOOGLE_DRIVE_FOLDER_ID e GOOGLE_DRIVE_SEND_ON.

11. Paralelismo, Selenium e Playwright

? Paralelo com Selenium: Zema, MadeiraMadeira e Probel.

? Magalu fora do pool paralelo, em rotina dedicada com Playwright.

? Webcontinental em fluxo HTTP/VTEX.

? Mercado Livre presente no codigo, mas documentado como desativado do projeto principal atual.

? Variaveis principais: ENABLE_PARALLEL_SCRAPING, MAX_WORKERS, MAX_SELENIUM_SESSIONS, SELENIUM_REMOTE_URL, SELENIUM_BROWSERS, SELENIUM_DRIVER_MAX_TASKS e bloco de configuracao Magalu.

12. Instalacao e Execucao

12.1 Dependencias Python declaradas

pandas

selenium

openpyxl

webdriver-manager

openai

playwright

requests

fastapi
uvicorn
jinja2
python-multipart
streamlit
tenacity
pydantic-settings
pandera
google-api-python-client
google-auth

12.2 Instaladores disponiveis

- ? install.py: instalador unificado cross-platform.
- ? install.ps1: instalacao recomendada em Windows.
- ? install.sh e install.command: wrappers Linux/macOS.

12.3 Modos de execucao

ModoComandoEfeito

Web.\run.ps1 -InterfaceMode webSobe API e Streamlit.

Local.\run.ps1 -InterfaceMode localSobe API e abre console local.

Both.\run.ps1 -InterfaceMode bothSobe API, Streamlit e console local.

13. Seguranca e Credenciais

- ? API_TOKEN para proteger consumo remoto da API.
- ? Login de usuarios no Streamlit e no painel admin.
- ? CRUD de usuarios com perfil admin ou operator.
- ? Persistencia controlada de usuarios em runs/admin_users.json.
- ? Configuracoes sensiveis lidas de .env.

Atencao: a documentacao final nao deve registrar credenciais reais, senhas temporarias ou tokens efetivos do ambiente. O objetivo e preservar conhecimento tecnico sem comprometer seguranca futura.

14. Deploy em Servidor Linux

- ? Artefatos em deploy/server/ para systemd, Nginx e healthcheck.
- ? Recomendacao de Python 3.11 ou 3.13 para deploy Linux.
- ? Pontos obrigatorios: API_TOKEN forte, PASSWORD_AUTH_ENABLED=1, senha bootstrap alterada, API_BASE e APP_PUBLIC_BASE_URL corretos.
- ? Preservar runs/, input.xlsx e arquivo de ambiente do servidor.

15. Publicacao Externa: Situacao Atual

Durante a fase de suporte recente, foram avaliadas estrategias de compartilhamento externo da UI sem dominio proprio. O Tailscale Funnel mostrou configuracao local, mas nao se consolidou como acesso externo utilizavel na internet publica para o caso analisado. A frente de compartilhamento foi pausada por decisao operacional, ate escolha de uma alternativa mais previsivel como Cloudflare Tunnel, ngrok, dominio proprio com proxy ou deploy em servidor dedicado.

16. Operacao do Dia a Dia

- Garantir Python e ambiente virtual funcional.
- Executar install.ps1 ou install.py quando necessario.
- Revisar .env com as configuracoes necessarias.
- Subir o sistema com run.ps1.
- Abrir a UI Streamlit ou usar o console local.
- Disparar um job manual e acompanhar o status.
- Baixar o output ou aguardar entrega automatica configurada.

Invoke-WebRequest http://127.0.0.1:8000/api/health

Invoke-WebRequest http://127.0.0.1:8000/api/overview

Invoke-WebRequest http://127.0.0.1:8000/api/daily/latest

.\run.ps1

.\run.ps1 -InterfaceMode local

python tools/local_console.py --api-base http://127.0.0.1:8000

17. Riscos, Cuidados e Recomendacoes Futuras

- ? Coletores de e-commerce sao sensiveis a mudancas de DOM e bloqueios anti-bot.
- ? Ambientes Windows podem perder referencia do Python-base e quebrar venv antigos.
- ? Publicacao externa da UI exige atencao com autenticacao, SSE e origem do iframe.
- ? Criar rotina formal de backup para runs/, input.xlsx e configuracoes de servidor.
- ? Formalizar uma estrategia unica de publicacao externa em vez de manter multiplas abordagens experimentais.
- ? Adicionar testes automatizados para endpoints principais e fluxos de validacao de API_BASE.

18. Encerramento

Este documento foi consolidado para servir como memoria tecnica permanente do projeto. Ele descreve nao apenas o estado atual do codigo, mas tambem as decisoes estruturais que transformaram o Agente de Precos em uma plataforma operacional completa, com backend, painel, console local, automacao, agenda e distribuicao de resultados.

Fim da documentacao consolidada em 18/03/2026.